Technical Assessment for IT024: NASA CMR AI Agent

Project Overview

Create an AI agent system that interfaces with NASA's Common Metadata Repository (CMR) API using natural language commands, with advanced capabilities for complex data discovery workflows.

Core Technical Requirements

Framework Selection & Architecture

- **Primary Framework**: Implement using LangGraph.
- **Multi-Agent Architecture**: Design a system with specialized agents for different tasks:
  - Query interpretation and validation agent
  - CMR API interaction agent
  - Data analysis and recommendation agent
  - Response synthesis and formatting agent
- **LLM Integration**: Support multiple LLM providers with fallback mechanisms
- **Scalability**: Design for concurrent request handling

Core Functionality

Intelligent Query Processing

- **Intent Classification**: Distinguish between exploratory, specific data requests, and analytical queries
- **Query Decomposition**: Break complex multi-part questions into sub-queries
- **Context Awareness**: Maintain conversation state and reference previous interactions
- **Query Validation**: Verify feasibility before API calls and suggest alternatives for invalid out-of-context requests

API Integration

- **Parallel Processing**: Execute Parallel agents when possible.
- **Multi API use**: Use results across collections, granules, and variables (e.g. Intent identification -> Collection search -> Granule search)
- **Metadata Enrichment**: Augment API responses with derived insights and recommendations

Required Features

1. Agentic Data Discovery Pipeline

User Query → Intent Analysis → Query Planning → API Orchestration → Result Synthesis → Response Generation

- **Smart Parameter Inference**: Automatically derive temporal/spatial bounds from natural language
- **Dataset Relationship Mapping**: Identify complementary datasets and data fusion opportunities
- **Quality Assessment**: Evaluate dataset completeness, temporal coverage, and resolution trade-offs

2. Intelligent Query Capabilities

- **Cross-collection Discovery**: Find related datasets across what's available
- **Constraint Optimization**: Balance user requirements with data availability, API response, context out-of-bounds, etc.

3. Performance & Reliability

- **Asynchronous Processing**: Handle long-running queries without blocking
- **Circuit Breakers**: Implement failure detection and recovery mechanisms
- **Streaming:** Use streaming output where possible

4. Data Analysis Integration (Visual Element)

- **Statistical Summaries**: Provide coverage statistics, temporal ranges, and spatial extents

Example Query

Example: "Compare precipitation datasets suitable for drought monitoring in Sub-Saharan Africa between 2015-2023, considering both satellite and ground-based observations, and identify gaps in temporal coverage."

Requirements:

- Parse multiple constraints (region, time, data type, research purpose)
- Execute parallel searches for augmented queries
- Perform gap analysis and coverage assessment
- Generate comparative recommendations

Example: "What datasets would be best for studying the relationship between urban heat islands and air quality in megacities?"

Requirements:

- Understand research domain relationships
- Suggest multi-disciplinary dataset combinations
- Recommend analysis methodologies
- Identify potential confounding variables

Evaluation Criteria

Technical Excellence (60%)

- **Code Architecture**: Clean separation of concerns, SOLID principles, design patterns
- **Performance Optimization**: Sub-second response times for simple queries, efficient resource usage
- **Error Resilience**: Comprehensive error handling, graceful degradation
- **Testing Strategy**: Unit tests, integration tests, performance tests, chaos engineering

Intelligence & Accuracy (40%)

- **Query Understanding**: Correct interpretation of complex, ambiguous queries
- **Result Relevance**: Precision and recall in dataset recommendations
- **Context Retention / Consistency**: Consistent behavior across conversation turns
- **Knowledge Application**: Appropriate use of Earth science domain knowledge

Bonus Challenge

- **Vector Database Integration**: Semantic search over NASA dataset documentation
- **Dynamic Context Retrieval**: Fetch relevant documentation based on query context
- **Knowledge Graph Construction**: Build relationships between datasets, instruments, and phenomena